

Python
for DA

Summary Session 10



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Экспресс-опрос



Что такое EDA?



Исследовательский анализ данных (EDA)

это метод анализа наборов данных для понимания их основных характеристик. Он включает в себя обобщение характеристик данных, выявление закономерностей и взаимосвязей с помощью визуальных и статистических методов.

Этапы проведения EDA



Понимание структуры и характеристик набора данных

На этом этапе EDA мы пристально изучаем структуру и характеристики данных, чтобы полностью понять, с чем имеем дело. Изучаем размер набора данных, типы переменных, выявляем пропущенные значения, дубликаты и другие важные аспекты.

Выявление аномалий и выбросов

Иногда данные могут хранить в себе сюрпризы, которые могут существенно повлиять на наши выводы. Используя методы EDA мы обнаруживаем аномалии и выбросы в наших данных.

Этапы проведения EDA



Идентификация связей и корреляций между переменными

С помощью статистических характеристик EDA позволяет выявить взаимосвязи между переменными, понять, как одни факторы влияют на другие.

Подготовка данных для дальнейших этапов анализа

На этом этапе EDA мы обрабатываем данные для того, чтобы подготовить их к более сложным аналитическим методам. Мы чистим данные от шума, заполняем пропущенные значения, проводим масштабирование или преобразования переменных, чтобы обеспечить их качественную и интерпретируемую структуру.

Основные этапы EDA

-  Загрузка и первоначальная проверка данных
-  Очистка и преобразование данных
-  Одномерный анализ
-  Двумерный анализ
-  Визуализация

Этапы EDA на примере



Рассмотрим основные этапы, возьмем набор данных с информацией о поддержанных автомобилях. Цель – на основе этих данных построить модель для прогнозирования цен поддержанных автомобилей. Само по себе создание модели – отдельный сложный процесс определения вклада каждого фактора в финальную цену. Но прежде чем приступить к этому процессу, нужно понять, какие факторы имеют влияние на цену, а какие вообще не нужно, нельзя или невозможно включить в модель. Собственно, это и есть основной результат, который мы хотели бы получить в результате исследовательского анализа данных.

Подключение модулей

```
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns
```

Загрузка данных

[Данные](#) для загрузки

```
data = pd.read_csv("used_cars_data.csv")
```

Получение общего представления о данных

```
data.head()
```

```
data.tail()
```

```
data.info()
```

Результат

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7253 entries, 0 to 7252
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   S.No.                 7253 non-null   int64
1   Name                  7253 non-null   object
2   Location              7253 non-null   object
3   Year                  7253 non-null   int64
4   Kilometers_Driven     7253 non-null   int64
5   Fuel_Type             7253 non-null   object
6   Transmission          7253 non-null   object
7   Owner_Type            7253 non-null   object
8   Mileage               7251 non-null   object
9   Engine                7207 non-null   object
10  Power                 7207 non-null   object
11  Seats                 7200 non-null   float64
12  New_Price             1006 non-null   object
13  Price                 6019 non-null   float64
dtypes: float64(2), int64(3), object(9)
memory usage: 793.4+ KB
```

Количество отсутствующих значений

```
data.isnull().sum()
```

Количество уникальных и дублированных значений

```
data.nunique()
```

```
data[data.duplicated()]
```


Экспресс-опрос

?

Что такое преобразование данных?



Преобразование данных

включает в себя их сокращение, корректировку и дополнение на основе имеющихся.

Исключение данных

```
data = data.drop(['S.No.'], axis = 1)  
data.info()
```

Преобразование необработанных данных

```
from datetime import date  
date.today().year  
data['Car_Age']=date.today().year-data['Year']  
data.head()
```

Преобразование необработанных данных

```
data['Brand'] = data.Name.str.split().str.get(0)
data['Model'] = data.Name.str.split().str.get(1) +
data.Name.str.split().str.get(2)
data.sample(10)
```

Анализ соответствия данных типам столбцов

```
print(data.Brand.unique())
```

```
['Maruti' 'Hyundai' 'Honda' 'Audi' 'Nissan' 'Toyota' 'Volkswagen' 'Tata'
 'Land' 'Mitsubishi' 'Renault' 'Mercedes-Benz' 'BMW' 'Mahindra' 'Ford'
 'Porsche' 'Datsun' 'Jaguar' 'Volvo' 'Chevrolet' 'Skoda' 'Mini' 'Fiat'
 'Jeep' 'Smart' 'Ambassador' 'Isuzu' 'ISUZU' 'Force' 'Bentley'
 'Lamborghini' 'Hindustan' 'OpelCorsa']
```

Исправление ошибок

```
data["Brand"] = data["Brand"].replace({"ISUZU": "Isuzu", "Mini": "Mini  
Cooper", "Land": "Land Rover"})
```

```
del data['Name']
```

Анализ ошибок с Mileage, Engine, Power, New_Price

```
data['Mileage_num'] = data.Mileage.str.split().str.get(0)
data['Mileage_units'] = data.Mileage.str.split().str.get(1)
data['Mileage_num'] = data['Mileage_num'].astype(float)
```


Просмотр результата

```
print(data.isnull().sum())  
print(data.Mileage_units.unique())  
print(data.info())
```

Унификация единиц

```
data['Mileage'] = data['Mileage_num']*(1+0.333*(data['Mileage_units']=='km/kg'))  
del data['Mileage_num']  
del data['Mileage_units']  
print(data.info())
```



ВОПРОСЫ



Цели этапа изучения данных



Проверка выбросов, закономерностей и тенденций в данных



Нахождение значимых закономерностей в данных



Получение углубленной информации о наборах данных для решения наших бизнес-задач



Определение принципа заполнения пропущенных значений в наборе данных.

Вывод описательных статистик

```
data.describe()
```

```
data.describe(include='all').T
```

	count	mean	std	min	25%	50%	75%	max
Year	7253.0	2013.365366	3.254421	1996.0000	2011.00	2014.00	2016.00	2.019000e+03
Kilometers_Driven	7253.0	58699.063146	84427.720583	171.0000	34000.00	53416.00	73000.00	6.500000e+06
Mileage	7251.0	18.224336	4.779547	0.0000	15.26	18.19	21.10	4.470882e+01
Engine	7207.0	1616.573470	595.285137	72.0000	1198.00	1493.00	1968.00	5.998000e+03
Power	7078.0	112.765214	53.493553	34.2000	75.00	94.00	138.10	6.160000e+02
Seats	7200.0	5.279722	0.811660	0.0000	5.00	5.00	5.00	1.000000e+01
New_Price	1003.0	21.339409	23.531486	2.7472	7.88	11.48	25.52	3.680000e+02
Price	6019.0	9.479468	11.187917	0.4400	3.50	5.64	9.95	1.600000e+02
Car_Age	7253.0	10.634634	3.254421	5.0000	8.00	10.00	13.00	2.800000e+01

Экспресс-опрос

?

Что такое одномерный анализ?



Одномерный анализ

этот анализ проводится методами визуализации. Категориальные переменные можно визуализировать с помощью столбчатых и круговых диаграмм. Числовые переменные можно визуализировать с помощью гистограмм, диаграмм “ящик с усами”, диаграмм плотности и др.

Распределение данных в каждом столбце

```
cat_cols=data.select_dtypes(include=['object']).columns.tolist()
num_cols = data.select_dtypes(include=np.number).columns.tolist()
print("Categorical Variables:")
print(cat_cols)
print("Numerical Variables:")
print(num_cols)
```


Визуализация числовых переменных

```
for col in num_cols:
    print(col)
    plt.figure(figsize = (15, 4))
    plt.subplot(1, 2, 1)
    data[col].hist(grid=False)
    plt.ylabel('count')
    plt.subplot(1, 2, 2)
    sns.boxplot(x=data[col])
    plt.show()
```

Визуализация категориальных переменных

```
fig, axes = plt.subplots(3, 2, figsize = (18, 18))
for i in range(1, len(cat_cols)):
    sns.countplot(ax = axes[(i-1)//2, (i-1)%2], x = cat_cols[i], data = data,
color = 'blue')
axes[0][0].tick_params(labelrotation=45)
axes[2][0].tick_params(labelrotation=90)
axes[2][1].tick_params(labelrotation=90)
plt.show()
```

Экспресс-опрос

?

Что такое логарифмическое преобразование?



Логарифмическое преобразование

статистический метод, применяемый для преобработки данных, который помогает улучшить их аналитическую обрабатываемость. Преобразование заключается в применении логарифма к каждому значению переменной.

Логарифмическое преобразование

```
for colname in ['Kilometers_Driven', 'Price']:  
    data[colname + '_log'] = np.log1p(data[colname])
```

Экспресс-опрос

?

Что такое двумерный анализ?



Двумерный анализ

Двумерный анализ помогает понять, как независимые переменные связаны друг с другом, а также как независимые переменные влияют на зависимые. Для числовых столбцов двумерный анализ выполняется с помощью парных графиков и диаграмм рассеяния. Для категориальных переменных используются столбчатые диаграммы разных типов.

Построение парного графика

```
plt.figure(figsize=(13,17))  
sns.pairplot(data)  
plt.show()
```


Сокращение столбцов

```
plt.figure(figsize=(13,17))  
sns.pairplot(data[['Kilometers_Driven_log', 'Mileage', 'Engine', 'Power',  
                  'Seats', 'Price_log', 'Car_Age']])  
plt.show()
```

Построение столбчатых диаграмм

```
fig, axarr = plt.subplots(3, 2, figsize=(8, 12))
for i in range(1, len(cat_cols)):
    data.groupby(cat_cols[i])['Price_log'].mean().\
    head(20).plot.bar(ax=axarr[(i-1)//2][(i-1)%2])
plt.subplots_adjust(hspace=0.8)
plt.show()
```

Построение столбчатых диаграмм

```
fig, axarr = plt.subplots(1, 2)
data.groupby('Seats')['Price_log'].mean().plot.bar(ax=axarr[0])
data.groupby('Car_Age')['Price_log'].mean().plot.bar(ax=axarr[1])
plt.show()
```

Экспресс-опрос

?

Что такое многомерный анализ?



Многомерный анализ

исследует более двух переменных для определения взаимосвязей между ними и анализа закономерностей. Чаще всего для этих целей используется тепловая карта, показывающая корреляцию между переменными.

Построение тепловой карты

```
plt.figure()  
sns.heatmap(data[['Kilometers_Driven_log', 'Kilometers_Driven', 'Mileage',  
                  'Engine', 'Power', 'Seats', 'Price_log', 'Price', 'Car_Age']].corr(), annot =  
            True)  
plt.show()
```

Способы заполнения пропущенных значений

-  Заполнение средним
-  Заполнение медианным
-  Заполнение наиболее встречающимся
-  Заполнение нулевым значениям
-  Использование сложных интерполяционных алгоритмов

Убрать нулевые значения

```
data.loc[data["Mileage"]==0.0, 'Mileage']=np.nan  
data.Mileage.isnull().sum()
```


Заполнение средним арифметическим

```
data['Mileage'] = data['Mileage'].fillna(value=np.mean(data['Mileage']))
```

Заполнение пропущенных значений

```
median_values = data.groupby(['Model', 'Brand'])['Seats'].transform('median')  
data.loc[data['Seats'].isnull(), 'Seats'] = data['Seats'].fillna(median_values)
```

Подведение итогов



Не существует определенных или идеальных правил для заполнения пропущенных значений в наборе данных. Каждый метод может работать лучше для одних наборов данных, но может работать хуже для других. Только практика и эксперименты позволяют понять, что работает лучше.

Выводы по проведенному анализу



Большинство клиентов предпочитают двухместные автомобили, поэтому цена на такие автомобили выше.



Цена автомобиля снижается с увеличением его возраста.



Клиенты предпочитают покупать автомобиль у первого владельца, а не у второго или третьего.



Из-за повышения цен на топливо клиенты предпочитают покупать электромобили.



Автоматическая коробка передач проще в использовании, чем механическая, поэтому более востребована.



ВОПРОСЫ



Экспресс-опрос



Что такое временной ряд?



Временной ряд

Это последовательность точек данных, собранных или записанных в последовательные моменты времени, как правило, через равномерные интервалы. Характеризуется временной упорядоченностью, что означает, что порядок наблюдения точек данных имеет решающее значение для анализа.

Отличия данных временных рядов



Данные временных рядов отличаются от других типов данных, так как они фиксируют изменение переменной во времени, что позволяет выявить тенденции, закономерности и сезонные колебания

Важность временных рядов

Финансы

Прогнозирование
погоды

Анализ продаж

- Отслеживания движения цен
- Прогнозирования
- Анализ волатильности

Важность временных рядов

Финансы

Прогнозирование
погоды

Анализ продаж

- Прогнозирование погодных условий
- Изучение климата
- Управление энергопотреблением

Важность временных рядов

Финансы

Прогнозирование
погоды

Анализ продаж

- Прогнозировать будущие продажи
- Выявлять сезонные тенденции
- Оптимизировать операции

Экспресс-опрос



Что такое тренд?



Тренд

Это долгосрочное движение в данных, указывающее на общее увеличение или уменьшение с течением времени.

Экспресс-опрос



Что такое сезонность?



Сезонность

Это регулярные, предсказуемые изменения, повторяющиеся в течение определенных периодов, таких как ежедневные, ежемесячные или ежегодные циклы.

Экспресс-опрос



Что такое шум?



Шум

Это случайные колебания данных, которые не следуют определенным закономерностям.

Экспресс-опрос



Что такое стационарность?



Стационарность

Это свойство временного ряда, при котором его статистические характеристики, такие как среднее значение и дисперсия, постоянны во времени.

Применение в различных областях

Эконометрика

Здравоохранение

Инженерия



Тренд

Это долгосрочная динамика ряда.
Показывает общее направление, в котором
движутся данные с течением времени:
увеличивается, уменьшается или остается
неизменным.



Сезонность

Это регулярные колебания, которые происходят через определенные промежутки времени, например, ежедневно, ежемесячно или ежегодно. Сезонность часто обусловлена внешними факторами, такими как праздники, погодные условия или бизнес-циклы.



Шум

Это случайная вариация ряда, которую нельзя отнести к тренду или сезонности. Это непредсказуемая часть данных, часто вызванная непредвиденными событиями или ошибками измерений.

Экспресс-опрос

?

Что такое декомпозиция временного ряда?



Декомпозиция временного ряда

Это процесс разделения временного ряда на составляющие его компоненты. Можно реализовать с помощью аддитивных или мультипликативных моделей.

Методы декомпозиции

Аддитивная модель

Мультипликативная
модель

Аддитивная модель



Подходит, когда величина сезонности не зависит от уровня ряда.
Компоненты складываются вместе:

$$y(t) = \text{Уровень} + \text{Тренд} + \text{Сезонность} + \text{Шум}$$

Мультипликативная модель



Подходит, когда величина сезонности масштабируется с уровнем ряда.
Компоненты перемножаются между собой:

$$y(t) = \text{Уровень} \times \text{Тренд} \times \text{Сезонность} \times \text{Шум}$$

Пример с использованием предустановленных данных

Для демонстрации компонентов воспользуемся набором данных AirPassengers, который предустановлен в библиотеке statsmodels в Python.

Импорт библиотек и загрузка данных

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.datasets import get_rdataset

data = get_rdataset('AirPassengers', 'datasets').data
data.index = pd.date_range(start = "1949-01", periods = len(data.index), freq =
"ME").to_period()
data.index = data.index.to_timestamp()
data.head()
```

Разложение временного ряда

```
# Декомпозиция временного ряда
decomposition = seasonal_decompose(data['value'], model='multiplicative')

trend = decomposition.trend
seasonal = decomposition.seasonal
residual = decomposition.resid
```

Построение графика компонентов

```
plt.figure(figsize=(12, 8))

plt.subplot(411)
plt.plot(data['value'], label='Original')
plt.legend(loc='best')

plt.subplot(412)
plt.plot(trend, label='Trend')
plt.legend(loc='best')

plt.subplot(413)
plt.plot(seasonal, label='Seasonality')
plt.legend(loc='best')

plt.subplot(414)
plt.plot(residual, label='Residuals')
plt.legend(loc='best')

plt.tight_layout()
plt.show()
```


Полезно знать

Исходная серия

Исходные данные, показывающие количество авиапассажиров с течением времени.

Тренд

Долгосрочный рост числа пассажиров.



Сезонность

Повторяющаяся закономерность в течение каждого года, показывающая увеличение числа пассажиров в определенные месяцы.

Остатки

Оставшийся шум после удаления тренда и сезонности, представляющий собой случайные колебания.



Стационарность

Это свойство, при котором статистические характеристики ряда, такие как среднее значение, дисперсия и автокорреляция, остаются неизменными с течением времени. Процесс, порождающий временной ряд, не меняется с течением времени, что делает ряд предсказуемым и более простым для моделирования.

Типы стационарности

Строгая
стационарность

Слабая
стационарность

Важность стационарности

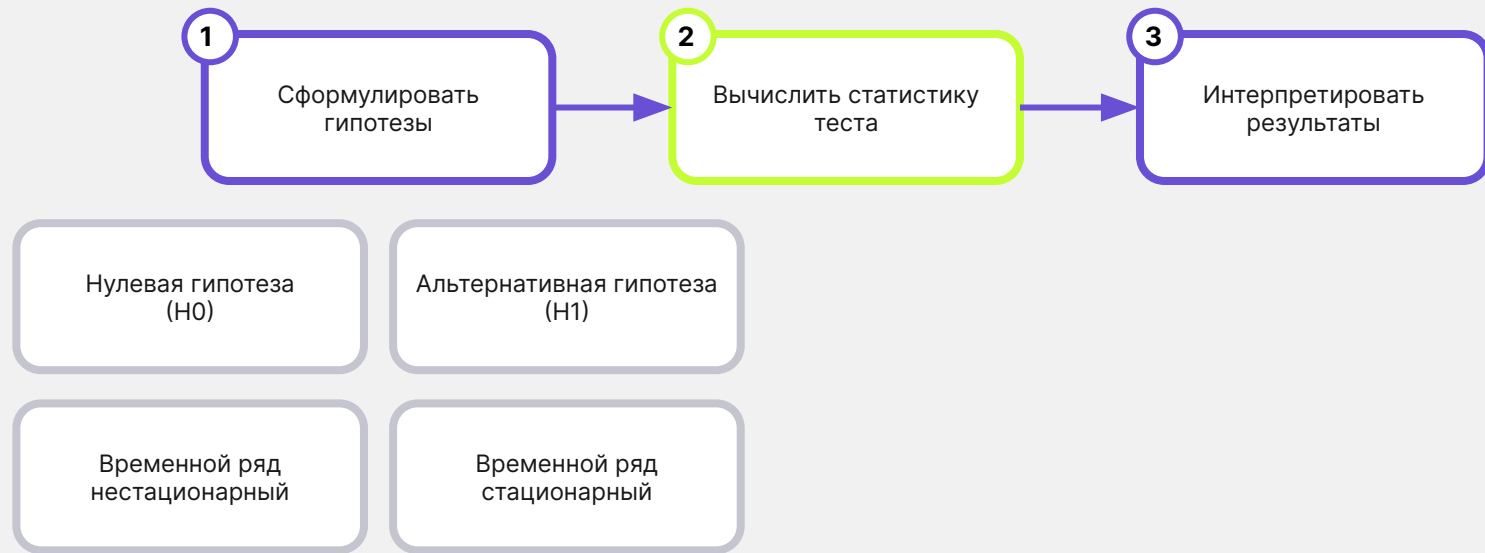
-  Предположения модели
-  Упрощенный анализ
-  Прогнозирование
-  Диагностические проверки
-  Улучшенная производительность моделей



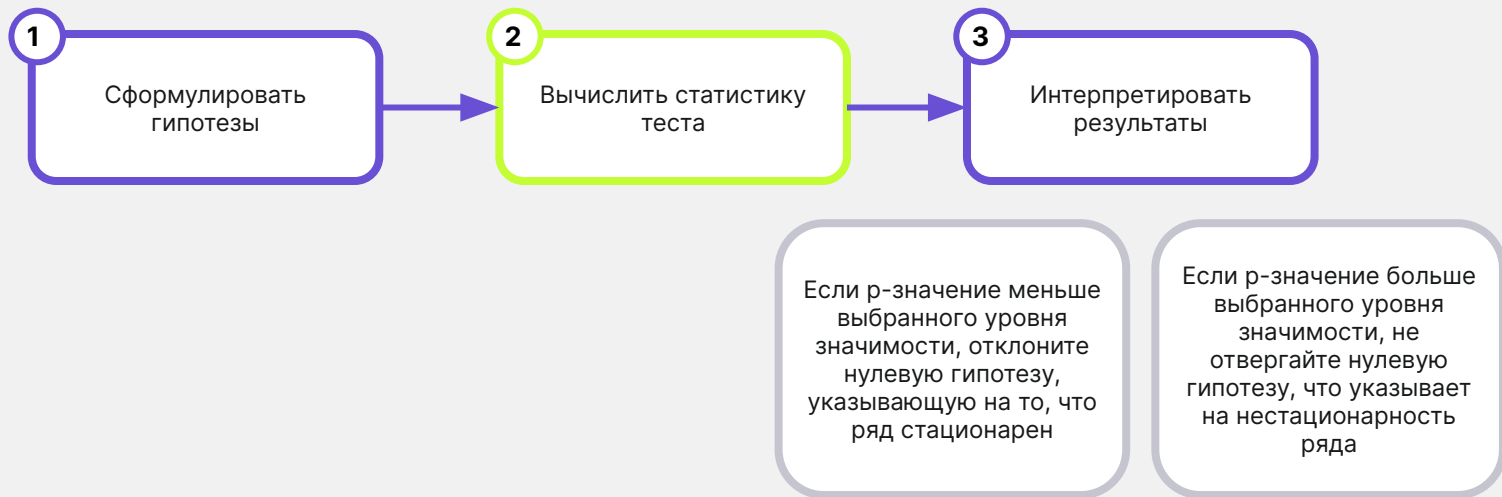
Дополненный тест Дикки-Фуллера (ADF)

Это распространенный статистический тест, используемый для определения стационарности временного ряда. Разновидность теста на единичный корень, который проверяет нулевую гипотезу о наличии единичного корня во временном ряду, что подразумевает нестационарность.

Шаги выполнения теста ADF



Шаги выполнения теста ADF



Пример с использованием предустановленных наборов данных

Для демонстрации теста ADF воспользуемся набором данных AirPassengers, который предустановлен в библиотеке statsmodels в Python.

Выполнение ADF-теста

```
from statsmodels.tsa.stattools import adfuller
result = adfuller(data['value'])
print('ADF Statistic:', result[0])
print('p-value:', result[1])
print('Критические значения:', result[4])
```

Интерпретация результатов

```
if result[1] < 0.05:  
    print("Временной ряд стационарен")  
else:  
    print("Временной ряд нестационарный")
```

Экспресс-опрос



Что такое автокорреляция?



Автокорреляция

Измеряет линейную связь между наблюдениями во временном ряду и наблюдениями на предыдущих временных шагах (лагах). Помогает выявить закономерности и зависимости в данных, указывая на то, как прошлые значения влияют на текущие.



Автокорреляционная функция (АКФ)

Оценивает корреляцию между наблюдениями для набора лагов.

Автокорреляция



$$ACF(k) = Corr(y_t, y_{t-k}),$$

где $Corr$ обозначает корреляционную функцию,
 k - лаг.

Интерпретация

Положительная АКФ

Указывает на положительную корреляцию между текущим наблюдением и наблюдением с лагом k

Отрицательная АКФ

Указывает на отрицательную корреляцию между текущим наблюдением и наблюдением с лагом k

Затухание АКФ

Постепенное затухание автокорреляции с увеличением лага часто указывает на наличие тренда или сезонности во временном ряду



Частичная автокорреляционная функция (PACF)

Помогает определить прямую связь между наблюдениями на определенных лагах, исключая вклад других лагов.

Пример с использованием предустановленных наборов данных

Воспользуемся набором данных `AirPassengers`, который предустановлен в библиотеке `statsmodels` в Python, чтобы продемонстрировать автокорреляцию и частичную автокорреляцию.

График автокорреляционной функции

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
plot_acf(data['value'], lags=50)
plt.title('Автокорреляционная функция (АКФ)')
plt.show()
```

График частичной автокорреляционной функции

```
plot_pacf(data['value'], lags=50)  
plt.title('Частичная автокорреляционная функция (PACF)')  
plt.show()
```

Экспресс-опрос

?

Что такое повторная выборка?



Повторная выборка

Это важнейшая техника анализа временных рядов, которая заключается в изменении частоты данных временного ряда. Это может быть сделано либо путем апсемплинга, либо путем даунсемплинга. Позволяет обобщать или агрегировать данные по различным временным периодам.

Экспресс-опрос



Что такое апсемплинг?



Апсемплинг

Это увеличение частоты данных временного ряда, например, преобразование ежедневных данных в часовые. При этом часто приходится заполнять недостающие значения с помощью таких методов, как интерполяция.

Экспресс-опрос

?

Что такое даунсемплинг?



Даунсемплинг

Это уменьшение частоты данных временного ряда, например, преобразование часовых данных в ежедневные. Обычно это связано с агрегированием точек данных с помощью таких функций, как `sum`, `mean` или `max`.

Пример демонстрации повторной выборки

Воспользуемся набором данных [Superstore sales](#). Этот набор данных содержит данные о продажах с временными метками, которые мы можем агрегировать до различных временных частот.

Импорт библиотек и загрузка данных

```
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_excel(r'Superstore.xlsx', sheet_name='Orders')
data['Order Date'] = pd.to_datetime(data['Order Date'])
data.set_index('Order Date', inplace=True)
```

Агрегация продаж по месяцам

```
# Агрегируем продажи по месяцам
monthly_sales = data['Sales'].resample('ME').sum()

# Постройте график продаж за месяц
plt.figure(figsize=(10, 6))
plt.plot(monthly_sales, label='Monthly Sales')
plt.title('Ежемесячные продажи магазина')
plt.xlabel('Дата')
plt.ylabel('Продажи')
plt.legend()
plt.show()
```

Агрегация продаж по неделям

```
# Агрегирование продаж по неделям
weekly_sales = data['Sales'].resample('W').sum()

# Постройте график недельных продаж
plt.figure(figsize=(10, 6))
plt.plot(weekly_sales, label='Weekly Sales')
plt.title('Еженедельные продажи в магазина')
plt.xlabel('Дата')
plt.ylabel('Продажи')
plt.legend()
plt.show()
```

Агрегация продаж по дням

```
# Агрегирование продаж по дням
daily_sales = data['Sales'].resample('D').sum()

# Постройте график ежедневных продаж
plt.figure(figsize=(10, 6))
plt.plot(daily_sales, label='Ежедневные продажи')
plt.title('Ежедневные продажи супермаркета')
plt.xlabel('Дата')
plt.ylabel('Продажи')
plt.legend()
plt.show()
```

Настройка агрегирования

```
# Агрегируем продажи по дням и вычисляем среднее значение
daily_sales_mean = data['Sales'].resample('D').mean()

# Постройте график среднедневных продаж
plt.figure(figsize=(10, 6))
plt.plot(daily_sales_mean, label='Среднедневные продажи')
plt.title('Среднедневные продажи в супермаркете')
plt.xlabel('Дата')
plt.ylabel('Средние продажи')
plt.legend()
plt.show()
```



ВОПРОСЫ



Экспресс-опрос



Что такое прогнозирование?



Прогнозирование

Это инструмент, используемый в различных областях для принятия обоснованных решений, планирования будущего и управления рисками.

Цель прогнозирования



Выявить закономерности и тенденции в исторических данных и использовать эту информацию для предсказания будущих результатов

Важность прогнозирования

-  Принятие обоснованных решений
-  Управление ресурсами
-  Финансовое планирование
-  Управление рисками
-  Операционная эффективность
-  Стратегическое планирование

Области применения

Финансы

Бизнес

Здравоохранение

Наука об
окружающей
среде

Энергетика

Анализ фондового
рынка

Курсы валют

Процентные ставки

Области применения

Финансы

Бизнес

Здравоохранение

Наука об
окружающей
среде

Энергетика

Прогнозирование
продаж

Управление
цепочками поставок

Маркетинг

Области применения

Финансы

Бизнес

Здравоохранение

Наука об
окружающей
среде

Энергетика

Вспышки заболеваний

Прием пациентов в
больницах

Распределение
ресурсов

Области применения

Финансы

Бизнес

Здравоохранение

Наука об
окружающей
среде

Энергетика

Прогнозирование
погоды

Климатическое
моделирование

Качество воздуха

Области применения

Финансы

Бизнес

Здравоохранение

Наука об
окружающей
среде

Энергетика

Прогнозирование
нагрузки на
электросети

Возобновляемые
источники энергии

Экспресс-опрос

?

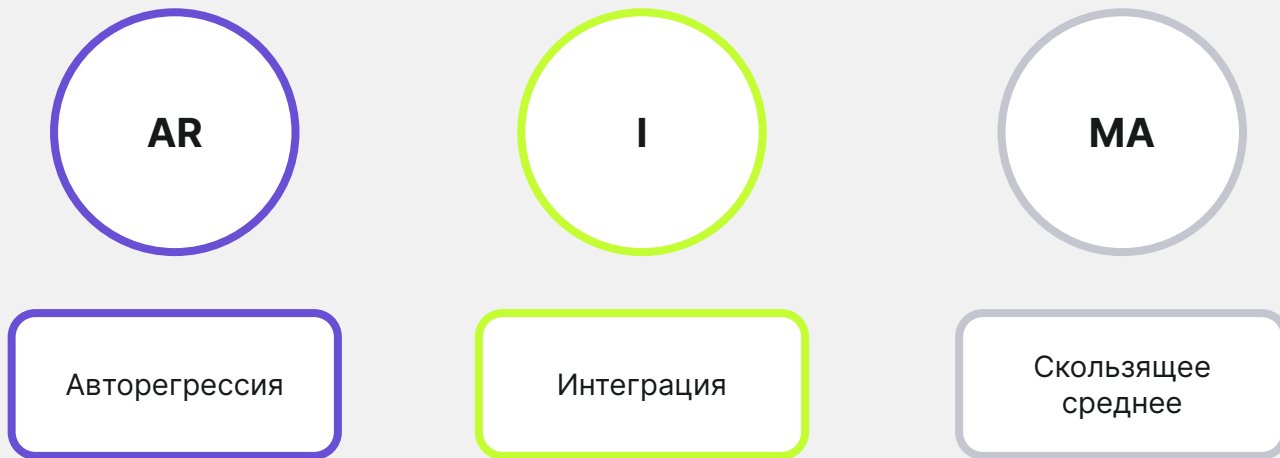
Что такое модель ARIMA?



Модель ARIMA

Или AutoRegressive Integrated Moving Average - это мощный и универсальный инструмент для прогнозирования временных рядов.

Модель ARIMA



Компонент авторегрессии (AR)



Определение

Авторегрессионный компонент модели ARIMA отражает связь между наблюдением и рядом запаздывающих наблюдений.

Параметр (p)

Количество запаздывающих наблюдений, включенных в модель. Показывает, сколько прошлых значений используется для прогнозирования текущего значения.

Компонент авторегрессии (AR)



Обозначения

AR(p) обозначает модель авторегрессии порядка p .

Пример

В модели AR(1) текущее значение y_t основывается на непосредственно предшествующем значении y_{t-1} . В модели AR(2) текущее значение y_t основывается на двух предыдущих значениях y_{t-1} и y_{t-2} .



Стационарность

Означает, что статистические свойства ряда, такие как среднее значение и дисперсия, постоянны во времени.

Экспресс-опрос

?

Что такое дифференцирование?



Дифференцирование

Это процесс вычитания предыдущего наблюдения из текущего.

Интегральная (I) компонента



Определение

Интегральная компонента предполагает дифференцирование временного ряда, чтобы сделать его стационарным.

Параметр (d)

Количество раз, которое необходимо продифференцировать данные для достижения стационарности.

Интегральная (I) компонента



Обозначения

$I(d)$ обозначает степень дифференцирования.

Пример

Если $d=1$, ряд дифференцируется один раз.
Если $d=2$, то ряд дифференцируется дважды.

Компонент скользящего среднего (МА)



Определение

Компонент скользящего среднего отражает связь между наблюдениями и остаточной ошибкой модели скользящего среднего, примененной к запаздывающим наблюдениям.

Параметр (p)

Количество запаздывающих ошибок прогноза в уравнении прогнозирования. Указывает на то, сколько прошлых членов ошибки используется для прогнозирования текущего значения.

Компонент скользящего среднего (МА)



Обозначения

МА(q) обозначает модель скользящего среднего порядка q .

Пример

В модели МА(1) текущее значение y_{t-1} основывается на члене ошибки, полученном на предыдущем временном шаге e_{t-1} . В модели МА(2) текущее значение y_t основывается на условиях ошибки двух предыдущих временных шагов e_{t-1} и e_{t-2} .

ARIMA(p, d, q)

p

Количество наблюдений с лагом (часть авторегрессии)

d

Количество дифференцированных данных (интегральная часть)

q

Количество запаздывающих ошибок прогноза (часть скользящего среднего)

Пример

Модель ARIMA(2, 1, 1) может быть выражена как:

$$y_t = \alpha + \beta_1 y_{t-1} + \beta_2 y_{t-2} + \theta_1 e_{t-1} + e_t,$$

где y_t - это текущее значение;

α - постоянная величина;

β_1 и β_2 - коэффициенты для наблюдений с запаздыванием;

θ_1 - коэффициент для запаздывающего термина ошибки;

e_t - ошибка в момент времени t .

Экспресс-опрос

?

Что такое statsmodels?



statsmodels

Это библиотека, которая предоставляет обширный набор инструментов для подгонки моделей ARIMA.

Шаги для подгонки модели ARIMA



Импорт библиотек и загрузка данных



Подгоните модель ARIMA



Проверить результаты

Подгонка модели ARIMA

```
from statsmodels.tsa.arima.model import ARIMA

# Загрузка данных
data = get_rdataset('AirPassengers', 'datasets').data
data.index = pd.date_range(start = "1949-01", periods = len(data.index), freq =
"ME").to_period()
data.index = data.index.to_timestamp()

# Определите модель ARIMA
model = ARIMA(data['value'], order=(5, 1, 0))

# Подгонка модели
model_fit = model.fit()

# Выведите итоговое значение модели
print(model_fit.summary())
```

Результаты

```

=====
                        SARIMAX Results
=====
Dep. Variable:          value    No. Observations:          144
Model:                 ARIMA(5, 1, 0)    Log Likelihood          -689.067
Date:                 Mon, 03 Jun 2024    AIC                   1390.135
Time:                 18:07:34    BIC                   1407.912
Sample:              01-01-1949    HQIC                  1397.358
                        - 12-01-1960
Covariance Type:      opg
=====
              coef    std err          z      P>|z|      [0.025    0.975]
-----
ar.L1         0.3223     0.097     3.334     0.001     0.133     0.512
ar.L2        -0.2170     0.078    -2.776     0.006    -0.370    -0.064
ar.L3        -0.0646     0.071    -0.915     0.360    -0.203     0.074
ar.L4        -0.2641     0.075    -3.519     0.000    -0.411    -0.117
ar.L5         0.0250     0.094     0.267     0.790    -0.159     0.209
sigma2       893.7229   113.383     7.882     0.000   671.497   1115.949
=====
Ljung-Box (L1) (Q):          0.00    Jarque-Bera (JB):          0.45
Prob(Q):                    0.97    Prob(JB):              0.80
Heteroskedasticity (H):      8.15    Skew:                  0.12
Prob(H) (two-sided):         0.00    Kurtosis:              3.14
=====

```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

Оценка модели ARIMA

Метрики

Средняя квадратичная
ошибка (MSE)

Корневая средняя
квадратичная ошибка
(RMSE)

Экспресс-опрос



Что такое MSE?



Средняя квадратичная ошибка (MSE)

Измеряет среднее значение квадратичной разницы между предсказанными и фактическими значениями.

Средняя квадратичная ошибка (MSE)



$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2,$$

где y_i - это фактическое значение в момент времени i ,
 $\hat{y}_i$ - это прогнозируемое значение в момент времени i ,
 n - общее количество наблюдений.

Средняя квадратичная ошибка (MSE)



Преимущества

- Штрафует большие ошибки
- Простота

Недостатки

- Зависимость от масштаба
- Интерпретируемость



Корень средней квадратичной ошибки (RMSE)

Это квадратный корень из MSE. Он позволяет оценить среднюю величину ошибок в тех же единицах, что и исходные данные.

Корень средней квадратичной ошибки (RMSE)



$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Корень средней квадратичной ошибки (RMSE)



Преимущества

- Интерпретируемость
- Штрафует большие ошибки

Недостатки

- Зависимость от масштаба
- Чувствительность к выбросам

Пример: оценка модели ARIMA с помощью MSE и RMSE

На примере набора данных AirPassengers продемонстрируем, как построить модель ARIMA и оценить ее эффективность с помощью MSE и RMSE.

Разделение данных

```
# Удаляем невостробованный столбец и делим данные на обучающий и тестовый наборы
del data['time']
train_size = int(len(data) * 0.8)
train, test = data[0:train_size], data[train_size:]
```

Подгонка модели ARIMA

```
# Подбор модели ARIMA  
model = ARIMA(train, order=(5, 1, 0))  
model_fit = model.fit()
```

Создание предсказаний

```
# Сделать прогнозы  
predictions = model_fit.forecast(steps=len(test))
```


Расчет MSE и RMSE

```
from sklearn.metrics import mean_squared_error
from math import sqrt
# Вычислить MSE
mse = mean_squared_error(test, predictions)
print('Среднеквадратичная ошибка:', mse)

# Вычислить RMSE
rmse = sqrt(mse)
print('Root Mean Squared Error:', rmse)
```

Построение графика результатов

```
# Постройте график сравнения фактических и прогнозируемых значений
plt.figure(figsize=(10, 6))
plt.plot(test.index, test, label='Actual')
plt.plot(test.index, predictions, label='Predicted', color='red')
plt.title('Фактические и прогнозируемые значения')
plt.xlabel('Дата')
plt.ylabel('Количество пассажиров')
plt.legend()
plt.show()
```

Экспресс-опрос

?

Что такое SARIMA?



SARIMA

Это расширение модели ARIMA, которая явно моделирует сезонный компонент в одномерных данных временных рядов.

SARIMA(p, d, q)(P, D, Q, s)

- p: порядок несезонной авторегрессии
- d: порядок несезонного дифференциала
- q: порядок несезонного скользящего среднего
- P: Порядок сезонной авторегрессии
- D: Порядок сезонного дифференцирования
- Q: Порядок сезонного скользящего среднего
- s: Длина сезонного цикла

Области использования SARIMA

-  Экономика
-  Финансы
-  Розничная торговля
-  Прогнозирование погоды
-  Энергетика
-  Здравоохранение

Пример с использованием предустановленного набора данных

Воспользуемся набором данных `AirPassengers`, который предустановлен в библиотеке `statsmodels`, чтобы продемонстрировать, как построить модель SARIMA.

Подгонка модели SARIMA

```
from statsmodels.tsa.statespace.sarimax import SARIMAX
# Определяем модель SARIMA
model = SARIMAX(data, order=(1, 1, 1), seasonal_order=(1, 1, 1, 12))

# Подгонка модели
model_fit = model.fit(dispatch=False)

# Выведите итоговое значение модели
print(model_fit.summary())
```


Создание предсказаний

```
# Сделать прогнозы
forecast = model_fit.forecast(steps=12)

# Постройте график результатов
plt.figure(figsize=(10, 6))
plt.plot(data, label='Actual')
plt.plot(forecast, label='Forecast', color='red')
plt.title('Прогноз количества авиапассажиров')
plt.xlabel('Дата')
plt.ylabel('Количество пассажиров')
plt.legend()
plt.show()
```

Экспресс-опрос

?

Что такое Facebook Prophet?



Facebook Prophet

Это библиотека с открытым исходным кодом, предназначенная для прогнозирования временных рядов данных.

Области Prophet

-  Аддитивная модель
-  Автоматическое прогнозирование
-  Настраиваемые прогнозы
-  Надежность

Модель Prophet



$$y(t)=g(t)+s(t)+h(t)+\varepsilon_t,$$

где $g(t)$ представляет собой компонент тренда

$s(t)$ представляет сезонный компонент

$h(t)$ представляет собой эффект праздников

ε_t - ошибка

Пример с использованием предустановленного набора данных

Воспользуемся набором данных `AirPassengers`, который предустановлен в библиотеке `statsmodels`, чтобы продемонстрировать, как подгонять модель `Prophet` и строить прогнозы.

Подгонка модели Prophet

```
from prophet import Prophet

data.reset_index(inplace=True)
data.columns = ['ds', 'y'] # Prophet требует, чтобы столбцы были названы 'ds' (дата) и
'y' (значение).

# Создаем экземпляр Prophet
model = Prophet()

# Подгонка модели к данным
model.fit(data)
```

Создание будущих предсказаний

```
# Создаем фрейм данных для хранения прогнозов
future = model.make_future_dataframe(periods=12, freq='M') # Прогнозирование на 12
месяцев вперед

# Делаем прогнозы
forecast = model.predict(future)
```


Визуализация прогноза

```
# Построить график прогноза
fig1 = model.plot(forecast)
plt.title('Прогноз количества авиапассажиров')
plt.xlabel('Дата')
plt.ylabel('Количество пассажиров')
plt.show()
```

```
# Построить график компонентов прогноза
fig2 = model.plot_components(forecast)
plt.show()
```



ВОПРОСЫ





ЗАДАНИЕ



Задание

Завершить работу над задачами с практикумов 19 и 20.



Задание 1



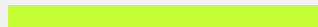
Скачайте [файл](#). Изучите датасет.

Задание 2



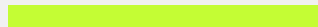
Выполните первичный анализ датасета (количество строк и столбцов, значения и типы данных, наличие отсутствующих значений, наличие категориальных столбцов, наличие дубликатов строк). Исправьте выявленные ошибки заполнения датасета. При необходимости выполните другие преобразования.

Задание 3



Добавьте столбец Sum с суммой всех баллов

Задание 4



Выполните одномерный анализ всех столбцов, включая добавленный

Задание 5

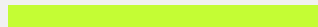
Выполните двумерный анализ влияния каждого фактора на столбец Sum

Задание 6



Выполните многомерный анализ с помощью цветовой карты корреляционной матрицы.

Задание 7



Заполните пропущенные значения наиболее подходящим способом.

Задание 1

Используя датасет PRSA_Data_Dingling_20130301-20170228.csv по [ссылке](#), разложите временной ряд на составляющие: тренд, сезонность и шум.

1. Загрузите набор данных и выберите данные о концентрации PM2.5.
2. Разложите временной ряд с помощью функции `seasonal_decompose` из `statsmodels`.
3. Постройте график исходного временного ряда и его разложенных компонентов.

Задание 2



Используя датасет PRSA_Data_Dingling_20130301-20170228.csv по [ссылке](#), определите, является ли временной ряд стационарным, используя тест дополненного Дикки-Фуллера (ADF).

1. Загрузите набор данных и постройте временной ряд концентрации PM2.5.
2. Выполните тест ADF, используя функцию adfuller из statsmodels.
3. Интерпретируйте результаты и определите, является ли ряд стационарным.

Задание 3

Используя датасет PRSA_Data_Dingling_20130301-20170228.csv по [ссылке](#), проведите анализ автокорреляции временного ряда с помощью автокорреляционной функции (ACF).

1. Загрузите набор данных и постройте временной ряд концентрации PM2.5.
2. Постройте график АКФ с помощью функции `plot_acf` из `statsmodels`.
3. Интерпретируйте график ACF, чтобы выявить значительные запаздывания и закономерности.

Задание 4



Используя датасет PRSA_Data_Dingling_20130301-20170228.csv по [ссылке](#), агрегирование данных временных рядов к различным временным частотам.

1. Загрузите набор данных и выберите данные о концентрации PM2.5.
2. Повторно дискретизируйте данные до ежедневной и еженедельной частоты.
3. Постройте временные ряды с повторной дискретизацией.

Задание 5



Используя датасет PRSA_Data_Dingling_20130301-20170228.csv по [ссылке](#), подгоните модель ARIMA к временному ряду и сделайте прогноз.

1. Загрузите набор данных и постройте временной ряд концентрации PM2.5.
2. Настройте модель ARIMA, используя класс ARIMA из statsmodels.
3. Сделайте прогноз на будущее и постройте график прогноза.

Задание 6

Используя датасет PRSA_Data_Dingling_20130301-20170228.csv по [ссылке](#), оценка эффективности модели ARIMA с помощью средней квадратичной ошибки (MSE) и корневой средней квадратичной ошибки (RMSE).

1. Загрузите набор данных и разделите его на обучающий и тестовый наборы.
2. Настройте модель ARIMA на обучающий набор и сделайте прогнозы на тестовом наборе.
3. Рассчитайте MSE и RMSE, чтобы оценить эффективность модели.

Задание 7



Используя датасет PRSA_Data_Dingling_20130301-20170228.csv по [ссылке](#), используйте Facebook Prophet для прогнозирования временного ряда.

1. Загрузите набор данных и подготовьте его для Prophet.
2. Подгоните модель Prophet к данным.
3. Сделайте прогнозы на будущее и постройте график прогноза.

Задание 8



Скачайте [датасет](#) с данными о среднем числе пятен на солнце ежемесячно с 1749 по 2021 годы.
Для этого датасета повторите задания 1-7.



ВОПРОСЫ





РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

1. Повторите первичный анализ датасета [used_cars_data](#) после всех преобразований, сделанных на уроке. Определите, типы каких столбцов теперь можно изменить. Сделайте это.
2. Повторите одномерный анализ прологорифмированных столбцов и столбцов, в которых были заполнены пропущенные значения.
3. Выведите информацию о самых крайних выбросах всех столбцов. Проанализируйте, есть ли среди них ошибки ввода. Ликвидируйте их любым способом, учитывающим специфику данных

Домашнее задание

Используйте датасет PRSA_Data_Dingling_20130301-20170228.csv по [ссылке](#).

Объедините прогнозы, полученные с помощью моделей ARIMA, SARIMA и Prophet, чтобы повысить точность предсказаний.

Инструкции:

1. Загрузите набор данных и выберите данные о концентрации PM2.5.
2. Агрегируйте данные по месяцам.
3. Сделайте прогноз на год с помощью моделей ARIMA, SARIMA и Prophet.
4. Объедините прогнозы всех трех моделей (возьмите их среднее арифметическое) и постройте график комбинированного прогноза.



ВОПРОСЫ



Заключение

